

ORCHARD

Final Production Arsenal

Gap Closure, Agent Operating System, Repository Contract, and Production Tooling

Prepared for orchard.cc

Purpose: convert the research manuals into a controlled production program.

Rule: no vague prototype language, no uncontrolled agent edits, no unsupported platform claims.

Output: a repo, a docs system, a release train, and a strict agent workflow.

1. Executive Decision

The current Orchard plan is strong enough to begin controlled repository creation, but it is not complete enough to begin broad feature implementation. The remaining work is not more brainstorming. The remaining work is specification closure: contracts, tests, safety gates, release operations, and agent controls.

The correct next move is to create the Orchard repository as the source of truth, install strict agent context files, open tracked work items, and begin the Foundation track. The first supported release must be narrow, testable, Apple-silicon-specific, and documented from the first commit.

Decision	Final stance
Core language	Swift. Use Swift Package Manager, Swift Concurrency, Foundation, OSLog, and macOS-native APIs.
Runtime strategy	RuntimeAdapter boundary first. Start with Apple container CLI adapter only behind protocol; move to Containerization API after source-backed proof.
Agent strategy	Use both Codex and Claude. Codex implements and runs tests. Claude reviews architecture, safety, docs, and high-risk diffs.
Tracking system	GitHub Issues/Projects is enough initially. Jira is acceptable if you want enterprise workflow discipline. Do not add process theater.
Event infrastructure	SQLite event table and OSLog locally. Do not use Confluent/Kafka in the core local control plane.
Release language	Use Foundation, Reconciliation, Networking, Security Hardening, Public Preview, Stable. Avoid prototype-style naming.

2. Remaining Gaps

There are still gaps. None of them invalidate the project, but ignoring them would create a fragile production cycle.

Gap	Why it matters	Required closure
Formal requirements	The docs describe direction, but implementation needs testable requirements.	Create requirements IDs: REQ-RUNTIME-001, REQ-RECON-001, REQ-SAFE-001, and link them to tests.
Acceptance tests	Agents need pass/fail contracts, not vibes.	Create a requirements-to-tests matrix before implementing runtime behavior.
Repository agent controls	Without context files, agents will drift and rewrite unsafe areas.	Install AGENTS.md, CLAUDE.md, CODEX.md, PR template, issue template, CODEOWNERS.
Swift package boundaries	A messy Swift package becomes impossible to evolve.	Create Sources modules before feature work. No direct shelling from reconciler or CLI logic.
Persistence schema	Desired state, observed state, events, and migrations need a stable data model.	Design SQLite schema, migrations, and crash recovery tests.
Release engineering	macOS tools require signing, packaging, upgrades, and eventually notarization.	Add packaging plan, release checklist, artifact signing, SBOM, and upgrade tests.
Apple Silicon performance lab	Optimization claims need measurements.	Add benchmark plan for boot latency, idle CPU, memory pressure, sleep/wake, and Rosetta warnings.

Network exposure policy	Ports, tunnels, DNS, and remote access create real security risk.	Add exposure classification: local-only, LAN, tunnel, cloud. Default to local-only.
Accelerator boundary proof	GPU/Metal/Core ML/MLX support cannot be claimed inside Linux VMs without proof.	Add a research task and proof matrix before any accelerator scheduling feature.
Human release authority	AI agents cannot approve high-risk infra changes alone.	Add two-agent gate plus human approval for high-risk folders.

3. How to Use the Existing Documents

The previous documents are not thrown away. They become source manuals for different tracks of work.

Document	Use it for	Do not use it for
Document 1: Agent Engineering Manual	Learning plan, repo philosophy, scaffolding, Kubernetes-style docs, RuntimeAdapter, reconciliation, architecture foundations.	Direct implementation without issue decomposition.
Document 2: Security and Apple Silicon Acceleration	Safety, secrets, supply chain, accelerator research, GPU/Metal/Core ML/MLX boundaries, resource policy.	Claiming supported accelerator behavior before proof.
Document 3: Network, Tunnels, Protocols, Cloud Security	vmnet, DNS, ports, tunnels, protocol boundaries, cloud exposure, network security.	Shipping remote tunnel functionality early.
This final document	Production cycle, remaining gaps, agent roles, tooling, markdown arsenal, issue tracking, release controls.	Replacing architecture docs or source-backed research.

4. Production Repository Structure

Create the repo with the following structure before implementation. Empty folders should contain a README explaining ownership and boundaries.

```
orchard/  
  README.md  
  LICENSE  
  SECURITY.md  
  GOVERNANCE.md  
  CODE_OF_CONDUCT.md  
  CONTRIBUTING.md  
  AGENTS.md  
  CLAUDE.md  
  CODEX.md  
  CODEOWNERS  
  Package.swift  
  
Sources/  
  OrchardCLI/  
  OrchardDaemon/  
  OrchardCore/  
  OrchardRuntime/  
  OrchardState/  
  OrchardReconciler/  
  OrchardHealth/  
  OrchardNetworking/
```

```

OrchardSecurity/
OrchardObservability/

Tests/
  OrchardCoreTests/
  OrchardRuntimeTests/
  OrchardStateTests/
  OrchardReconcilerTests/
  OrchardIntegrationTests/
  OrchardPerformanceTests/

docs/
  PROJECT_CHARTER.md
  getting-started/
  concepts/
  tasks/
  tutorials/
  reference/
  architecture/
  design/
  contributing/

examples/
  single-service/
  api-postgres/
  api-postgres-redis/
  health-restart/
  drift-demo/

schemas/
  orchard-yaml.schema.json

scripts/
  bootstrap.sh
  test.sh
  lint.sh
  format.sh
  doctor-dev.sh

.github/
  workflows/
  ISSUE_TEMPLATE/
  PULL_REQUEST_TEMPLATE.md

```

5. Swift Architecture Contract

The core rule is separation. CLI parses intent. Core validates state. Reconciler plans actions. RuntimeAdapter executes Apple container operations. State persists facts. No layer should quietly reach around another layer.

Module	Responsibility	Forbidden
OrchardCLI	Argument parsing, command dispatch, user output.	Runtime shell-out, SQLite writes, business logic.
OrchardDaemon	Long-running service, reconciliation scheduling, lifecycle.	Manifest parsing shortcuts or direct runtime logic.
OrchardCore	Domain types, validation models, manifest types, errors.	Apple container command execution.
OrchardRuntime	RuntimeAdapter protocol and Apple container adapters.	Desired-state planning.
OrchardState	SQLite persistence, migrations, state queries.	Runtime calls.
OrchardReconciler	Observe, diff, plan, apply coordination.	Direct process spawning or CLI output

Orchard Final Production Arsenal - orchard.cc

		parsing.
OrchardHealth	Health checks, backoff, restart policy.	Network exposure or cleanup ownership.
OrchardNetworking	Port allocation, network inspection, tunnel/exposure policy later.	Unapproved cloud tunnel behavior.
OrchardSecurity	Policy checks, destructive-action gates, mount validation, secret redaction.	Storing secrets in plaintext.
OrchardObservability	Events, logs, status rendering, diagnostics.	Secrets, raw credentials, unbounded telemetry.

6. Frameworks and Tools

Area	Use	Decision
CLI	Swift ArgumentParser	Use for command parsing and help output.
Concurrency	Swift Concurrency	Use async/await, Task cancellation, actors where state isolation matters.
Logging	OSLog / unified logging	Use for daemon and runtime diagnostics.
Persistence	SQLite	Use local durable state; begin with thin adapter or approved Swift wrapper.
Manifest parsing	YAML parser such as Yams	Allowed after dependency approval. Validate with JSON Schema in CI.
Testing	Swift Testing and XCTest	Use unit tests, mock RuntimeAdapter, integration tests gated on Apple container.
Formatting	swift-format	Required. SwiftLint optional if rules are stable.
Docs	Markdown + MkDocs Material or VitePress	Use one docs generator; prioritize simple versioned Markdown.
Diagrams	Mermaid	Use text-based diagrams checked into docs.
Security scanning	gitleaks, CodeQL, Semgrep, osv-scanner	Required CI gates.
Supply chain	Syft, SBOM, artifact signing, provenance	Required before public release artifacts.
Performance	Instruments, xctrace, powermetrics, vm_stat, memory_pressure	Required for Apple silicon optimization claims.
Issue tracking	GitHub Issues/Projects or Jira	GitHub is sufficient for solo or small team; Jira if using enterprise ceremonies.
Event streaming	SQLite events + OSLog	Confluent/Kafka is not needed in the local core.

7. Codex vs Claude: Final Agent Assignment

Use both. Do not ask one agent to both create and approve high-risk infrastructure changes.

Work type	Primary agent	Reviewer	Reason
Swift package scaffolding	Codex	Claude	Codex can edit/run local code and tests; Claude checks boundaries.
RuntimeAdapter	Codex	Claude + human	Low-level runtime calls

implementation			need executable testing and separate safety review.
Reconciler logic	Codex	Claude	Codex writes testable logic; Claude checks state-machine correctness.
Architecture docs and ADRs	Claude	Codex	Claude is stronger for large-context synthesis and critique; Codex checks repo consistency.
Security model	Claude	Codex + human	Threat modeling and destructive-operation review require adversarial reasoning.
CI and scripts	Codex	Claude	Codex handles runnable workflows; Claude checks policy gaps.
Network/tunnel work	Codex	Claude + human	Network exposure is high-risk; never merge single-agent.
Performance benchmark harness	Codex	Claude	Codex implements; Claude checks measurement design.

If only one tool has reliable local shell and test access on your Mac, that tool becomes the implementation agent by default. The review agent should still inspect diffs and design before merge.

8. Work Tracking and Enterprise Process

Do not overbuild process, but do create traceability. Every serious infra repo needs issues, decisions, tests, and releases connected.

Tool/process	Use now	Use later
GitHub Issues	Required. Track every engineering task.	Keep even if Jira is added.
GitHub Projects	Recommended. Simple board for Foundation, Reconciliation, Networking, Security Hardening.	May be replaced by Jira if team grows.
Jira	Optional. Use if you want enterprise-style epics, dependency tracking, and structured sprints.	Recommended when multiple people/agents are working in parallel.
Confluent/Kafka	No. Not for local core events.	Only for remote enterprise event ingestion after a cloud control-plane decision.
Slack/Discord	Optional notifications only.	Do not make chat the source of truth.
Google Docs	Research and review drafts only.	Final architecture must live in repo Markdown.
GitHub Actions	Required for lint, tests, docs, security scans.	Add macOS arm64 runner for runtime integration.
Release checklist	Required before public artifacts.	Add signing, notarization, SBOM, changelog, upgrade test.

9. Foundation Track: Ordered Work

1. Create repository with markdown arsenal and governance files.

Orchard Final Production Arsenal - orchard.cc

2. Create Swift package with empty modules and compile gate.
3. Add formatting, linting, unit test, docs, and security scan workflows.
4. Create RuntimeAdapter protocol and MockRuntimeAdapter only.
5. Create manifest domain types and validation errors.
6. Create SQLite schema draft and migration harness.
7. Create event model and redaction rules.
8. Create AppleContainerCLIAdapter spike in isolated branch.
9. Create doctor-check specification before doctor implementation.
10. Create requirements-to-tests matrix before reconciliation implementation.

10. Stop Conditions

Pause implementation if any of the following happen:

- Apple container behavior cannot be observed reliably enough for the RuntimeAdapter contract.
- The repo starts adding compatibility features before reconciliation is correct.
- A destructive operation lacks preview, ownership checks, or recovery path.
- A dependency is added because it is convenient rather than required.
- Network exposure is implemented before local-only service behavior is stable.
- Accelerator support is claimed without a source-backed, local proof.
- Agents modify high-risk code without a separate review gate.

11. Required Markdown Arsenal

The companion ZIP contains starting versions of these files. Put them in the repository before running coding agents on implementation tasks.

File	Purpose
AGENTS.md	Shared agent rules for all tools.
CLAUDE.md	Claude-specific architecture, review, and safety instructions.
CODEX.md	Codex-specific implementation, test, and completion rules.
docs/PROJECT_CHARTER.md	Source-of-truth charter and first supported release contract.
docs/architecture/dependency-policy.md	Allowed dependencies and review rules.
docs/architecture/agent-production-workflow.md	Two-agent workflow and merge gates.
docs/architecture/tooling-standard.md	Standardized tools, CI, scans, tracking, and event infrastructure.
SECURITY.md	Security policy and required checks.
GOVERNANCE.md	Decision and review model.
.github/PULL_REQUEST_TEMPLATE.md	PR review gate.
.github/ISSUE_TEMPLATE/engineering-task.md	Task shape for agents and humans.
CODEOWNERS	High-risk ownership map.

12. Final Direction

Begin repo creation now. Do not begin broad feature coding. The next production cycle is repository foundation, agent controls, Swift package boundaries, requirements IDs, test harnesses, and RuntimeAdapter protocol. Once those are stable, implement Apple container behavior behind the adapter with strict integration tests on Apple silicon.

The project becomes credible by being narrow, measurable, documented, and hard to break. The agents should be treated as workers under a production system, not as the production system itself.

13. Source Anchors to Keep in the Repo

- Apple container technical overview: <https://github.com/apple/container/blob/main/docs/technical-overview.md>
- Apple Containerization: <https://github.com/apple/containerization>
- Kubernetes controllers: <https://kubernetes.io/docs/concepts/architecture/controller/>
- Kubernetes CRI: <https://kubernetes.io/docs/concepts/containers/cri/>
- OpenAI Codex docs: <https://developers.openai.com/codex>
- OpenAI Codex CLI docs: <https://developers.openai.com/codex/cli>
- Claude Code overview: <https://code.claude.com/docs/en/overview>
- Claude Code memory/CLAUDE.md docs: <https://code.claude.com/docs/en/memory>